

Bài giảng về dùng Vibe Coding (dùng prompt) sinh đoạn mã và chương trình Làm việc với Tập hợp và Xác suất

✓ Colletted and Prepared Vibe Coding by Prof Happy AI, 2026

```
%matplotlib inline
```

prompt:Viết mã Python sử dụng thư viện SymPy để tìm Power Set của tập hợp {1, 2, 3}. Yêu cầu: Giải thích ngắn gọn cách hoạt động của hàm .powerset() và in ra tổng số tập hợp con thu được.

```
#P125: Finding the power set of a set
from sympy import FiniteSet
s = FiniteSet(1, 2, 3)
ps = s.powerset()
ps
```

```
{0, {1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}, {1, 2, 3}}
```

prompt: Sử dụng thư viện SymPy trong Python, hãy tạo hai tập hợp hữu hạn (FiniteSet): tập hợp s chứa {1, 2, 3} và tập hợp t chứa {2, 4, 6}. Sau đó, thực hiện phép hợp (union) của hai tập hợp này và in kết quả

```
#P126: Union of two Sets
from sympy import FiniteSet
s = FiniteSet(1, 2, 3)
t = FiniteSet(2, 4, 6)
s.union(t)
# {1, 2, 3, 4, 6}
```

```
{1, 2, 3, 4, 6}
```

prompt: Sử dụng thư viện SymPy trong Python, hãy tạo hai tập hợp hữu hạn (FiniteSet): tập hợp s chứa {1, 2, 3} và tập hợp t chứa {2, 4, 6}. Sau đó, thực hiện phép giao (intersection) của hai tập hợp này và in kết quả

```
#P127: Intersection of two Sets
from sympy import FiniteSet
s = FiniteSet(1, 2)
t = FiniteSet(2, 3)
s.intersect(t)
```

```
{2}
```

Prompt: Sử dụng thư viện SymPy trong Python, hãy tạo hai tập hợp hữu hạn FiniteSet là $s = \{1, 2\}$ và $t = \{3, 4\}$. Viết mã để tính tích Descartes của chúng bằng toán tử * và dùng vòng lặp for để in từng cặp phần tử trong kết quả ra màn hình.

```
#P127/128: Cartesian product of two Sets
from sympy import FiniteSet
s = FiniteSet(1, 2)
t = FiniteSet(3, 4)
p = s*t
for elem in p:
    print(elem)
```

```
(1, 3)
(2, 3)
(1, 4)
(2, 4)
```

prompt: Viết chương trình Python sử dụng thư viện SymPy để tính chu kỳ dao động của con lắc đơn. Yêu cầu cụ thể: Định nghĩa hàm time_period(length, g) sử dụng công thức $T = 2\pi\sqrt{\frac{L}{g}}$. Sử dụng FiniteSet để tạo tập hợp chiều dài $L = \{15, 18, 21, 22.5, 25\}$ (đơn vị cm) và tập hợp gia tốc trọng trường $g = \{9.8, 9.78, 9.83\}$. Tính tích Descartes của hai tập hợp này bằng toán tử * để thử nghiệm mọi tổ

hợp của L và g . Sử dụng vòng lặp để duyệt qua các tổ hợp, lưu ý đổi đơn vị L sang mét trước khi tính toán. In kết quả dưới dạng bảng có 3 cột: Length, Gravity, và Time Period, định dạng căn lề giữa và làm tròn kết quả thời gian 3 chữ số thập phân.

```
#P130: Different gravity, different results
from sympy import FiniteSet, pi
def time_period(length, g):
    T = 2*pi*(length/g)**0.5
    return T

if __name__ == '__main__':
    L = FiniteSet(15, 18, 21, 22.5, 25)
    g_values = FiniteSet(9.8, 9.78, 9.83)
    print('{0:^15}{1:^15}{2:^15}'.format('Length(cm)', 'Gravity(m/s^2)', 'Time Period(s)'))
    for elem in L*g_values:
        l = elem[0]
        g = elem[1]
        t = time_period(l/100, g)
        print('{0:^15}{1:^15}{2:^15.3f}'.format(float(l), float(g), float(t)))
```

Length(cm)	Gravity(m/s^2)	Time Period(s)
22.5	9.78	0.953
15.0	9.78	0.778
22.5	9.8	0.952
18.0	9.78	0.852
15.0	9.8	0.777
22.5	9.83	0.951
21.0	9.78	0.921
18.0	9.8	0.852
15.0	9.83	0.776
25.0	9.78	1.005
21.0	9.8	0.920
18.0	9.83	0.850
25.0	9.8	1.004
21.0	9.83	0.918
25.0	9.83	1.002

Prompt: Viết chương trình Python tính xác suất để một số nguyên tố xuất hiện khi tung xúc xắc 20 mặt. Yêu cầu:

Định nghĩa hàm probability(space, event) để tính tỉ lệ giữa số phần tử của biến cố và không gian mẫu.

Định nghĩa hàm check_prime(number) để kiểm tra một số có phải là số nguyên tố hay không (lưu ý xử lý trường hợp số 1).

Sử dụng thư viện SymPy và FiniteSet để tạo không gian mẫu (sample space) từ 1 đến 20.

Duyệt qua không gian mẫu để lọc các số nguyên tố và đưa vào một FiniteSet biến cố (event).

In ra không gian mẫu, tập hợp các số nguyên tố tìm được và xác suất cuối cùng với định dạng 5 chữ số thập phân.

```
#P132: Probability of a Prime number appearing when a 20-sided dice is rolled
def probability(space, event):
    return len(event)/len(space)

def check_prime(number):
    if number != 1:
        for factor in range(2, number):
            if number % factor == 0:
                return False
    else:
        return False
    return True

if __name__ == '__main__':
    space = FiniteSet(*range(1, 21))
    primes = []
    for num in space:
        if check_prime(num):
            primes.append(num)
    event= FiniteSet(*primes)
    p = probability(space, event)
    print('Sample space: {0}'.format(space))
    print('Event: {0}'.format(event))
    print('Probability of rolling a prime: {0:.5f}'.format(p))
```

```
Sample space: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
Event: {2, 3, 5, 7, 11, 13, 17, 19}
Probability of rolling a prime: 0.40000
```

Prompt: Sử dụng thư viện SymPy trong Python, hãy viết chương trình tính xác suất để biến cố A hoặc biến cố B xảy ra. Cụ thể:

Tạo không gian mẫu s là một FiniteSet chứa các số từ 1 đến 6.

Tạo tập hợp biến cố a chứa {2, 3, 5} và biến cố b chứa {1, 3, 5}.

Tính biến cố hợp e (biến cố A hoặc B) bằng phương thức .union().

Tính và in ra xác suất theo công thức độ dài của tập hợp biến cố chia cho độ dài của không gian mẫu bằng f-string."

```
#P134: Probability of event A or event B
from sympy import FiniteSet
s = FiniteSet(1, 2, 3, 4, 5, 6)
a = FiniteSet(2, 3, 5)
b = FiniteSet(1, 3, 5)
e = a.union(b)
print(f"Probability of event A or event B: {len(e)/len(s)}")
```

Probability of event A or event B: 0.6666666666666666

prompt:Sử dụng thư viện SymPy trong Python, hãy viết chương trình tính xác suất để cả hai biến cố A và B cùng xảy ra. Cụ thể:

Khai báo không gian mẫu s là một FiniteSet gồm các số từ 1 đến 6.

Khai báo tập hợp biến cố a = {2, 3, 5} và b = {1, 3, 5}.

Tìm biến cố giao e (A and B) bằng phương thức .intersect().

Tính xác suất bằng tỉ lệ len(e)/len(s) và in kết quả bằng f-string với thông báo: 'Probability of event A and event B

```
#P134: Probability of event A and event B
from sympy import FiniteSet
s = FiniteSet(1, 2, 3, 4, 5, 6)
a = FiniteSet(2, 3, 5)
b = FiniteSet(1, 3, 5)
e = a.intersect(b)
print(f" Probability of event A and event B {len(e)/len(s)}")
```

Probability of event A and event B 0.3333333333333333

prompt: Viết chương trình Python mô phỏng việc tung xúc xắc 6 mặt cho đến khi tổng điểm đạt ít nhất 20. Yêu cầu cụ thể:

Import thư viện random và matplotlib.pyplot (dù chưa dùng đến plot trong bước này).

Định nghĩa một hàm roll() trả về giá trị ngẫu nhiên từ 1 đến 6.

Trong khối if **name** == 'main', thiết lập biến target_score = 20.

Sử dụng vòng lặp while để tung xúc xắc liên tục, cộng dồn điểm vào biến score và đếm số lần tung vào biến num_rolls.

Mỗi lần tung, in ra giá trị của xúc xắc bằng .format().

Khi kết thúc vòng lặp, in ra thông báo tổng điểm đạt được và tổng số lần tung đã thực hiện

```
#P135: Can you Roll that score?

'''
Roll a die until the total score is 20
'''

import matplotlib.pyplot as plt
import random

target_score = 20
def roll():
    return random.randint(1, 6)

if __name__ == '__main__':
    score = 0
    num_rolls = 0
    while score < target_score:
        die_roll = roll()
        num_rolls += 1
        print('Rolled: {}'.format(die_roll))
        score += die_roll
    print('Score of {} reached in {} rolls'.format(score, num_rolls))
```

```

Rolled: 3
Rolled: 3
Rolled: 6
Rolled: 6
Rolled: 2
Score of 20 reached in 5 rolls

```

prompt: Để tạo ra đoạn mã tính xác suất đạt được điểm mục tiêu khi tung xúc xắc nhiều lần bằng cách sử dụng không gian mẫu của SymPy, bạn có thể sử dụng prompt sau:***Viết chương trình Python sử dụng thư viện SymPy để tính xác suất tổng điểm của n lần tung xúc xắc đạt tối thiểu một giá trị mục tiêu. Yêu cầu cụ thể: Định nghĩa hàm `find_prob(target_score, max_rolls)`. Sử dụng `FiniteSet(1, 2, 3, 4, 5, 6)` và toán tử lũy thừa `**` để tạo không gian mẫu s cho số lần tung `max_rolls`. Trong hàm, lọc các tổ hợp có tổng (sum) lớn hơn hoặc bằng `target_score` để tạo tập hợp biến cố thành công. Cần xử lý riêng trường hợp `max_rolls > 1` (duyệt qua các tuple) và `max_rolls = 1`. Trả về kết quả là tỉ lệ giữa số phần tử của tập biến cố và không gian mẫu. Trong khối `if name == 'main':`, cho người dùng nhập vào điểm mục tiêu và số lần tung tối đa từ bàn phím, sau đó in xác suất với định dạng 5 chữ số thập phân.

```

#P136: Is the target score possible?
from sympy import FiniteSet
import random
def find_prob(target_score, max_rolls):
    die_sides = FiniteSet(1, 2, 3, 4, 5, 6)
    # sample space
    s = die_sides**max_rolls
    # Find the event set
    if max_rolls > 1:
        success_rolls = []
        for elem in s:
            if sum(elem) >= target_score:
                success_rolls.append(elem)
    else:
        if target_score > 6:
            success_rolls = []
        else:
            success_rolls = []
            for roll in die_sides:
                if roll >= target_score:
                    success_rolls.append(roll)
    e = FiniteSet(*success_rolls)
    # calculate the probability of reaching target score
    return len(e)/len(s)
if __name__ == '__main__':
    target_score = int(input('Enter the target score: '))
    max_rolls = int(input('Enter the maximum number of rolls allowed: '))
    p = find_prob(target_score, max_rolls)
    print('Probability: {0:.5f}'.format(p))

```

```

Enter the target score: 6
Enter the maximum number of rolls allowed: 2
Probability: 0.72222

```

prompt:Viết chương trình Python mô phỏng một máy ATM giả định rút các loại tờ tiền có mệnh giá khác nhau với xác suất tùy biến. Yêu cầu cụ thể:

Định nghĩa hàm `get_index(probability)` nhận vào một danh sách các xác suất. Hàm này phải tính toán xác suất tích lũy (cumulative probability), sinh một số ngẫu nhiên từ 0 đến 1 và trả về chỉ số (index) tương ứng dựa trên số ngẫu nhiên đó.

Định nghĩa hàm `dispense()` chứa danh sách các mệnh giá tiền [5, 10, 20, 50] và danh sách xác suất tương ứng [1/6, 1/6, 1/3, 1/3]. Hàm này gọi `get_index` để quyết định mệnh giá nào được rút ra.

Trong khối `main`, thực hiện mô phỏng rút tiền 10.000 lần và lưu kết quả vào một danh sách.

Sử dụng thư viện `matplotlib.pyplot` để vẽ biểu đồ tần suất (histogram) hiển thị phân phối của các tờ tiền đã rút và đặt tiêu đề biểu đồ là 'Bill Dispensed'

```

#P139: Simulate a fictional ATM
'''
Simulate a fictional ATM that dispenses dollar bills
of various denominations with varying probability
'''

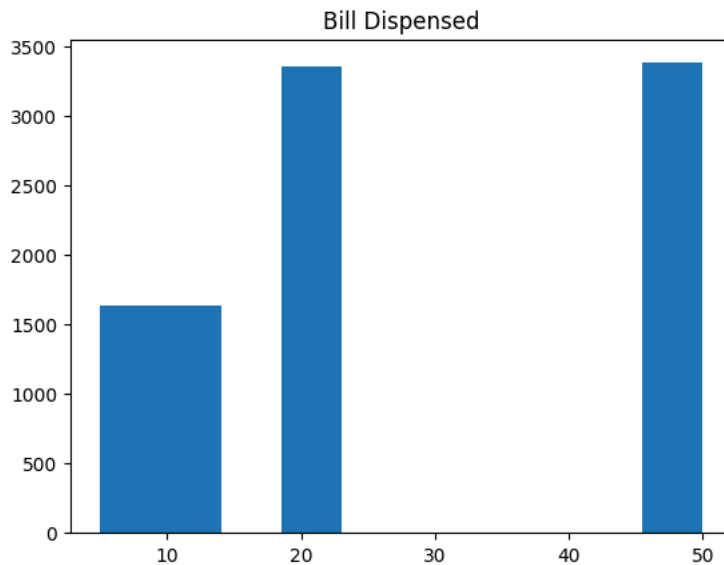
import random
import matplotlib.pyplot as plt

```

```
def get_index(probability):
    c_probability = 0
    sum_probability = []
    for p in probability:
        c_probability += p
        sum_probability.append(c_probability)
    r = random.random()
    for index, sp in enumerate(sum_probability):
        if r <= sp:
            return index
    return len(probability)-1

def dispense():
    dollar_bills = [5, 10, 20, 50]
    probability = [1/6, 1/6, 1/3, 1/3]
    bill_index = get_index(probability)
    return dollar_bills[bill_index]

# Simulate a large number of bill withdrawals
if __name__ == '__main__':
    bill_dispensed = []
    plt.title('Bill Dispensed')
    for i in range(10000):
        bill_dispensed.append(dispense())
    # plot a histogram
    plt.hist(bill_dispensed)
    plt.show()
```



prompt: Để tạo ra đoạn mã vẽ biểu đồ Venn cho hai tập hợp bằng cách kết hợp matplotlib-venn và SymPy như trên, bạn có thể sử dụng prompt sau:

***Viết chương trình Python để vẽ biểu đồ Venn cho hai tập hợp số. Yêu cầu cụ thể:

Sử dụng thư viện matplotlib_venn (hàm venn2) để vẽ biểu đồ và matplotlib.pyplot để hiển thị.

Sử dụng FiniteSet của thư viện SymPy để định nghĩa hai tập hợp:

Tập s1: Các số lẻ từ 1 đến 19.

Tập s2: Các số nguyên tố từ 2 đến 19.

Định nghĩa hàm draw_venn(sets) nhận vào một danh sách các tập hợp, thực hiện vẽ biểu đồ Venn và đặt tiêu đề biểu đồ là 'Venn Diagram'.

Trong khối if **name** == '**main**', khởi tạo hai tập hợp trên và gọi hàm để hiển thị kết quả.

```
#P140: Draw a Venn diagram for two sets
...

Draw a Venn diagram for two sets
...

from matplotlib_venn import venn2
import matplotlib.pyplot as plt
from sympy import FiniteSet
```

```
plt.title('Venn Diagram')
def draw_venn(sets):
    venn2(subsets=sets)
    plt.show()

if __name__ == '__main__':
    s1 = FiniteSet(1, 3, 5, 7, 9, 11, 13, 15, 17, 19)
    s2 = FiniteSet(2, 3, 5, 7, 11, 13, 17, 19)
    draw_venn([s1, s2])
```

Venn Diagram

